CSE 3230 • SOFTWARE DEVELOPMENT & WEB TECHNOLOGIES LAB

Testing & CI/CD Pipeline: Complete Study Guide

A line-by-line explanation of every Unit Test, Integration Test, and GitHub Actions CI workflow file

AUTHOR	ENVIRONMENT	TEST FRAMEWORKS	CI/CD ENGINE
Muttakin Rahman Shoscho	Java 21 • Spring Boot 3.5.16	JUnit 5, Mockito, MockMvc, H2	GitHub Actions (Ubuntu 24.04)

GUIDE TABLE OF CONTENTS

1. The Big Picture: Test Pyramid & Strategy
2. Testing Dependencies in `pom.xml`
3. Unit Tests: `AdminServiceTest.java`
4. Unit Tests: `StudentServiceTest.java`
5. Integration Tests: `TeacherIntegrationTest.java`
6. CI Pipeline: `.github/workflows/ci.yml`
7. Behind the Scenes: Core Mechanics & Invariants
8. Experimental Results & Benchmarks
9. 20 Likely Teacher Viva Questions & Answers
10. 1-Minute Lab Presentation Pitch

1. The Big Picture: Test Pyramid & Strategy

When writing software for production, manual testing through Swagger UI or Postman is slow and error-prone. Automated testing guarantees that whenever code changes, every business invariant and security constraint is verified in seconds.

In the University Management System (UMS), we structured our tests into the standard industry **Test Pyramid**:

```
  /\\ ← 3 Integration Tests (MockMvc + Spring Security + H2 DB) [~6s execution] / \\ Tests full HTTP request flow, status codes (201, 401, 409), transactions /====\\ / \\ ← 6 Unit Tests (JUnit 5 + Mockito mocks) [~0.22s execution] /========\\ Tests pure business invariants in isolation, zero database needed
```

UNIT TESTS What & Why

Target: `AdminService` and `StudentService`.
Technique: Mock dependencies with Mockito.
Speed: ~36 ms per test (0.22 s total).
Purpose: Validates edge cases like preventing deletion of the last admin or blocking duplicate student IDs without booting Spring.

INTEGRATION TESTS What & Why

Target: `TeacherController` + Security + H2 DB.
Technique: Spring `MockMvc` + in-memory database.
Speed: ~0.75 s for the suite (6 s with context boot).
Purpose: Validates the real HTTP pipeline: JSON shapes, BCrypt hashing, HTTP headers, and 401/409 responses.

2. Testing Dependencies in `pom.xml`

Testing requires specialized libraries defined in `pom.xml` with `<scope>test</scope>` so they never ship to the production Docker container:

Dependency	Key Packages Included	Role in Testing
`spring-boot-starter-test`	JUnit 5 (Jupiter), Mockito, AssertJ, Spring Test	The foundational testing starter. Provides `@Test`, `@ExtendWith`, `MockMvc`, and `assertThat(...)` assertions.
`spring-security-test`	`SecurityMockMvcRequestPostProcessors`	Enables injecting authentication into MockMvc calls: `.with(httpBasic("admin", "admin123"))` or `.with(user("...").roles("ADMIN"))`.

Dependency	Key Packages Included	Role in Testing
`com.h2database:h2`	H2 In-Memory RDBMS Engine	Creates an ephemeral SQL database in RAM during tests (`MODE=MySQL`). Zero Docker or MySQL setup needed on CI runners.

3. Unit Tests: `AdminServiceTest.java` (Line-by-Line Breakdown)

Location: `src/test/java/com/university/ums/service/AdminServiceTest.java`
This class contains **3 isolated unit tests** verifying administrative business rules.

Complete Source Code

```
package com.university.ums.service;

import com.university.ums.dto.AdminRequest;
import com.university.ums.dto.AdminResponse;
import com.university.ums.exception.ConflictException;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Admin;
import com.university.ums.repository.AdminRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.test.util.ReflectionTestUtils;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class AdminServiceTest {

    @Mock
    private AdminRepository adminRepository;

    @Mock
    private UserAccountService accounts;

    @InjectMocks
    private AdminService adminService;

    @Test
    void create_validAdmin_savesAndReturnsResponse() {
        AdminRequest request = new AdminRequest("Admin User", "admin@uni.edu", "01700000000", "admin_us
        Admin savedAdmin = new Admin();
        ReflectionTestUtils.setField(savedAdmin, "id", 10L);
        savedAdmin.setName("Admin User");
        savedAdmin.setEmail("admin@uni.edu");
        savedAdmin.setUsername("admin_user");

        when(adminRepository.save(any(Admin.class))).thenReturn(savedAdmin);

        AdminResponse response = adminService.create(request);

        assertThat(response.id()).isEqualTo(10L);
        assertThat(response.username()).isEqualTo("admin_user");
        verify(accounts).requirePassword("password123");
        verify(accounts).ensureUnique("admin_user", "admin@uni.edu", null);
        verify(adminRepository).save(any(Admin.class));
    }

    @Test
    void findById_nonExistingId_throwsResourceNotFoundException() {
        when(adminRepository.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> adminService.findById(99L))
                .isInstanceOf(ResourceNotFoundException.class)
                .hasMessageContaining("Admin with id 99 not found");
```

```
    }

    @Test
    void delete_lastAdmin_throwsConflictException() {
        Admin admin = new Admin();
        ReflectionTestUtils.setField(admin, "id", 1L);
        when(adminRepository.findById(1L)).thenReturn(Optional.of(admin));
        when(adminRepository.count()).thenReturn(1L);

        assertThatThrownBy(() -> adminService.delete(1L))
                .isInstanceOf(ConflictException.class)
                .hasMessage("Cannot delete the last admin account");

        verify(adminRepository, never()).delete(any());
    }
}
```

Line-by-Line Explanation of `AdminServiceTest`

- `@ExtendWith(MockitoExtension.class)` : Tells JUnit 5 to initialize Mockito before running the tests. It initializes all fields annotated with `@Mock` and injects them into `@InjectMocks`. It takes **0.05 seconds** to run because no Spring container is loaded.
- `@Mock private AdminRepository adminRepository` : Creates a Mockito proxy (fake) object for the repository. Any method call on it returns default values (null, 0, empty Optional) unless we stub it with `when(...).thenReturn(...)`.
- `@InjectMocks private AdminService adminService` : Mockito inspects `AdminService`'s constructor `public AdminService(AdminRepository, UserAccountService)` and passes the two mock objects into it.
- `ReflectionTestUtils.setField(savedAdmin, "id", 10L)` : The entity `AppUser` has `@Id @GeneratedValue` and therefore has no public `setId()` method. In unit tests where Hibernate is not running to generate the ID, we use Spring's `ReflectionTestUtils` to set the private field via reflection.
- `when(adminRepository.save(any())).thenReturn(savedAdmin)` : Defines the stub rule: whenever `adminRepository.save()` is called with any `Admin` object, return our fake `savedAdmin`.
- `verify(accounts).requirePassword("password123")` : Asserts that `AdminService.create()` actually called the password check. If someone removes that line in the service, this test fails immediately!
- `verify(adminRepository, never()).delete(any())` : In the last admin test, this asserts that the database delete was **never called**, confirming that the last admin is protected.

4. Unit Tests: `StudentServiceTest.java` (Line-by-Line Breakdown)

Location: `src/test/java/com/university/ums/service/StudentServiceTest.java`
This class contains **3 isolated unit tests** verifying student registration and uniqueness invariants.

Complete Source Code

```java
package com.university.ums.service;

import com.university.ums.dto.StudentRequest;
import com.university.ums.dto.StudentResponse;
import com.university.ums.exception.ConflictException;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Student;
import com.university.ums.repository.StudentRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.test.util.ReflectionTestUtils;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class StudentServiceTest {

    @Mock
    private StudentRepository studentRepository;

    @Mock
    private UserAccountService accounts;

    @InjectMocks
    private StudentService studentService;

    @Test
    void create_validStudent_savesAndReturnsResponse() {
        StudentRequest request = new StudentRequest("Student One", "student@uni.edu", "01711111111",
                "student_one", "password123", "2007001", "CSE", 2020);
        Student savedStudent = new Student();
        ReflectionTestUtils.setField(savedStudent, "id", 5L);
        savedStudent.setName("Student One");
        savedStudent.setEmail("student@uni.edu");
        savedStudent.setUsername("student_one");
        savedStudent.setStudentId("2007001");
        savedStudent.setDepartment("CSE");
        savedStudent.setBatch(2020);

        when(studentRepository.existsByStudentId("2007001")).thenReturn(false);
        when(studentRepository.save(any(Student.class))).thenReturn(savedStudent);

        StudentResponse response = studentService.create(request);

        assertThat(response.id()).isEqualTo(5L);
        assertThat(response.studentId()).isEqualTo("2007001");
        verify(accounts).requirePassword("password123");
        verify(accounts).ensureUnique("student_one", "student@uni.edu", null);
        verify(studentRepository).save(any(Student.class));
    }

    @Test
    void create_duplicateStudentId_throwsConflictException() {
```

```
        StudentRequest request = new StudentRequest("Student One", "student@uni.edu", "01711111111",
                "student_one", "password123", "2007001", "CSE", 2020);

        when(studentRepository.existsByStudentId("2007001")).thenReturn(true);

        assertThatThrownBy(() -> studentService.create(request))
                .isInstanceOf(ConflictException.class)
                .hasMessage("Student ID '2007001' is already registered");

        verify(studentRepository, never()).save(any());
    }

    @Test
    void findById_nonExistingId_throwsResourceNotFoundException() {
        when(studentRepository.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> studentService.findById(99L))
                .isInstanceOf(ResourceNotFoundException.class)
                .hasMessageContaining("Student with id 99 not found");
    }
}
```

Line-by-Line Explanation of `StudentServiceTest`

- `StudentRequest request = new StudentRequest(...)`: Instantiates the Java record representing the JSON body sent by the client. Contains specific student attributes like `studentId: "2007001"`, `department: "CSE"`, and `batch: 2020`.
- `when(studentRepository.existsByStudentId("2007001")).thenReturn(false)`: In Test 4, instructs Mockito that the student ID does not yet exist in the database, allowing registration to proceed.
- `when(studentRepository.existsByStudentId("2007001")).thenReturn(true)`: In Test 5, instructs Mockito that the student ID already exists. The service immediately aborts and throws `ConflictException` before attempting to save.
- `verify(studentRepository, never()).save(any())`: Asserts that on a duplicate student ID error, no corrupt or partial record is saved to the database.

5. Integration Tests: `TeacherIntegrationTest.java` (Line-by-Line Breakdown)

Location: `src/test/java/com/university/ums/TeacherIntegrationTest.java`
This class contains **3 full-slice integration tests** exercising HTTP endpoints through Spring Security, Jackson JSON, validation, and in-memory H2 SQL.

Complete Source Code

```
package com.university.ums;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;
import org.springframework.transaction.annotation.Transactional;

import static org.hamcrest.Matchers.containsString;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessor
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class TeacherIntegrationTest {

    @Autowired
    private MockMvc mvc;

    private static final String TEACHER_PAYLOAD = """
            {
                "name": "Dr. Hasan",
                "email": "hasan@uni.edu",
                "phone": "01722222222",
                "username": "hasan",
                "password": "teacherPass123",
                "department": "CSE",
                "designation": "Assistant Professor"
            }
            """;

    @Test
    void fullTeacherLifecycle_createReadUpdateDelete() throws Exception {
        // 1. Create Teacher as Admin
        MvcResult result = mvc.perform(post("/api/teachers")
                        .with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(TEACHER_PAYLOAD))
                .andExpect(status().isCreated())
                .andExpect(header().string("Location", containsString("/api/teachers/")))
                .andExpect(jsonPath("$.username").value("hasan"))
                .andExpect(jsonPath("$.department").value("CSE"))
                .andExpect(jsonPath("$.designation").value("Assistant Professor"))
                .andExpect(jsonPath("$.password").doesNotExist())
                .andReturn();

        String location = result.getResponse().getHeader("Location");

        // 2. Read created Teacher
        mvc.perform(get(location).with(httpBasic("admin", "admin123")))
```

```
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.email").value("hasan@uni.edu"));

        // 3. Update Teacher designation
        String updatePayload = """
                {
                    "name": "Dr. Hasan",
                    "email": "hasan@uni.edu",
                    "phone": "01722222222",
                    "username": "hasan",
                    "department": "CSE",
                    "designation": "Associate Professor"
                }
                """;
        mvc.perform(put(location)
                        .with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(updatePayload))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.designation").value("Associate Professor"));

        // 4. Delete Teacher
        mvc.perform(delete(location).with(httpBasic("admin", "admin123")))
                .andExpect(status().isNoContent());

        // 5. Verify Not Found
        mvc.perform(get(location).with(httpBasic("admin", "admin123")))
                .andExpect(status().isNotFound());
    }

    @Test
    void createTeacher_duplicateUsername_returnsConflict() throws Exception {
        mvc.perform(post("/api/teachers")
                        .with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(TEACHER_PAYLOAD))
                .andExpect(status().isCreated());

        String duplicatePayload = """
                {
                    "name": "Another Teacher",
                    "email": "another@uni.edu",
                    "phone": "01733333333",
                    "username": "hasan",
                    "password": "newPass123",
                    "department": "EEE",
                    "designation": "Lecturer"
                }
                """;

        mvc.perform(post("/api/teachers")
                        .with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(duplicatePayload))
                .andExpect(status().isConflict())
                .andExpect(jsonPath("$.message").value("Username 'hasan' is already taken"));
    }

    @Test
    void unauthenticatedAccess_isRejected() throws Exception {
        mvc.perform(get("/api/teachers"))
                .andExpect(status().isUnauthorized())
                .andExpect(jsonPath("$.status").value(401));
```

```
        mvc.perform(post("/api/teachers")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(TEACHER_PAYLOAD))
                .andExpect(status().isUnauthorized());
    }
}
```

Line-by-Line Explanation of `TeacherIntegrationTest`

- `@SpringBootTest`: Starts the Spring container, scans for all `@Service`, `@RestController`, and `@Repository` components, and creates the real security filter chain.
- `@AutoConfigureMockMvc`: Automatically configures the `MockMvc` helper. Instead of starting an HTTP server on port 8080 (which requires open sockets and can cause port conflicts), `MockMvc` feeds fake HTTP requests directly to Spring's `DispatcherServlet` in memory.
- `@Transactional`: In Spring Test, adding `@Transactional` to a test class wraps each test method in an SQL transaction. When the test finishes, Spring **rolls back the transaction**. Any teacher inserted during the test vanishes from the in-memory H2 database, leaving it 100% clean for the next test!
- `.with(httpBasic("admin", "admin123"))`: Adds an HTTP Basic `Authorization: Basic YWRtaW46YWRtaW4xMjM=` header to the request. Spring Security's `BasicAuthenticationFilter` intercepts this, authenticates with `CustomUserDetailsService`, and grants `ROLE_ADMIN`.
- `.andExpect(status().isCreated())`: Asserts that the HTTP response code is `201 Created`.
- `.andExpect(header().string("Location", containsString("/api/teachers/")))`: Asserts REST compliance—a `POST` resource creation must return the URI of the newly created resource in the `Location` header.
- `.andExpect(jsonPath("$.password").doesNotExist())`: Critical security assertion! Ensures the BCrypt hash is excluded from JSON serialization and never leaks to callers.
- `.andExpect(status().isUnauthorized())`: In Test 3, confirms that unauthenticated requests receive `401 Unauthorized` with a structured `ApiError` JSON body.

6. CI Pipeline: `.github/workflows/ci.yml` (Line-by-Line Breakdown)

Location: `.github/workflows/ci.yml`
Continuous Integration automatically runs on GitHub's cloud runners on every push. Below is every single line explained.

Complete Workflow File

```yaml
1: name: Java CI with Maven
2:
3: on:
4:   push:
5:     branches: [ "main" ]
6:   pull_request:
7:     branches: [ "main" ]
8:
9: jobs:
10:   build:
11:     runs-on: ubuntu-latest
12:
13:     steps:
14:     - name: Check out repository
15:       uses: actions/checkout@v5
16:
17:     - name: Set up JDK 21
18:       uses: actions/setup-java@v5
19:       with:
20:         java-version: '21'
21:         distribution: 'temurin'
22:         cache: maven
23:
24:     - name: Make Maven wrapper executable
25:       run: chmod +x mvnw
26:
27:     - name: Run tests with Maven
28:       run: ./mvnw -B test
```

Line-by-Line Explanation of the CI Workflow

Lines	YAML Snippet	Detailed Explanation
1	`name: Java CI with Maven`	Defines the title of the workflow displayed in the GitHub Actions tab.
3–7	`on: push / pull_request`	Specifies the event triggers. Whenever someone runs `git push` to `main` or opens a pull request targeting `main`, GitHub automatically launches this workflow.
9–11	`runs-on: ubuntu-latest`	Instructs GitHub to provision an isolated virtual machine running Ubuntu 24.04 LTS with 2 vCPUs and 7 GB of RAM.
14–15	`uses: actions/checkout@v5`	Clones our Git repository into the runner VM. We upgraded to **v5** to run natively on **Node.js 24** and eliminate all deprecation warnings.
17–22	`uses: actions/setup-java@v5`	Installs the exact Java SDK: • `java-version: '21'`: Matches our `pom.xml` `<java.version>21</java.version>`. • `distribution: 'temurin'`: Uses Eclipse Temurin, the official OpenJDK build.

Lines	YAML Snippet	Detailed Explanation
		• `cache: maven` : Caches the `~/.m2/repository` folder across runs, cutting build times from ~1 minute to **20–30 seconds**.
24–25	`run: chmod +x mvnw`	Ensures the Maven wrapper script (`mvnw`) has executable POSIX permissions (`rwxr-xr-x`) so Linux can execute it.
27–28	`run: ./mvnw -B test`	Runs all tests using the bundled Maven wrapper. The `-B` flag stands for *Batch Mode*: it suppresses ASCII animation bars and non-essential logs, producing clean deterministic output in CI logs.

7. Behind the Scenes: Core Mechanics & Invariants

1. How MockMvc Simulates Requests Without Tomcat

Starting embedded Tomcat and binding to port 8080 takes ~1.5 seconds and requires an open network socket. `MockMvc` builds a mock `HttpServletRequest` in RAM and calls Spring's `DispatcherServlet.service(req, res)` directly. It runs through every real Filter, Interceptor, Controller, and ExceptionHandler, giving 100% realistic HTTP testing without network overhead.

2. The Last Admin Invariant Check

In `AdminService.java` : `if (adminRepository.count() <= 1) throw new ConflictException("Cannot delete the last admin account");` . Because our system requires an administrator to create other administrators or modify roles, deleting the final admin would permanently lock the entire university out. Our unit test `delete_lastAdmin_throwsConflictException` guarantees this invariant can never be accidentally broken.

3. Resolving Entity ID Setting in Unit Tests

JPA entities inherit from `AppUser` with `@Id @GeneratedValue(strategy = IDENTITY) private Long id;` . There is intentionally no public setter to maintain entity immutability. In integration tests, Hibernate assigns this ID automatically during SQL insert. In unit tests, Hibernate is not running; we solve this elegantly with `ReflectionTestUtils.setField(admin, "id", 10L)` , which sets the private field directly via reflection.

8. Experimental Results & Benchmarks

Execution of `./mvnw test` runs **25 automated tests** (16 baseline API tests + 6 service unit tests + 3 integration tests). Every test passes with zero failures and zero errors:

Test Suite File	Type	Tests	Failures	Errors	Execution Duration
`AdminServiceTest.java`	Unit (Mockito)	3	0	0	**0.049 s**
`StudentServiceTest.java`	Unit (Mockito)	3	0	0	**0.167 s**
`TeacherIntegrationTest.java`	Integration (MockMvc + H2)	3	0	0	**0.783 s**
`UniversityApiTests.java`	Integration (MockMvc + H2)	16	0	0	**5.434 s**
Total Test Suite	**Combined**	**25**	**0**	**0**	**6.433 s**

9. 20 Likely Teacher Viva Questions & Answers

Q1: What is the difference between a unit test and an integration test in Spring Boot?

A: A unit test tests one class (like a Service) in complete isolation by mocking all dependencies with Mockito. An integration test boots the Spring ApplicationContext and tests multiple layers together (Controllers, Security, Services, Repositories, and the database).

Q2: Why use `@ExtendWith(MockitoExtension.class)` instead of `@SpringBootTest` for service tests?

A: `@SpringBootTest` boots the entire Spring container and Hibernate, taking 2–5 seconds. `@ExtendWith(MockitoExtension.class)` only initializes Mockito, running in 0.05 seconds. Unit tests should be extremely fast so developers run them continuously.

Q3: What does Mockito's `when(...).thenReturn(...)` do?

A: It sets up a "stub rule". It tells the mock object: "When this specific method is called with these arguments, return this canned value instead of executing real code."

Q4: What does Mockito's `verify(...)` do?

A: It checks whether a specific method was actually called on a mock object, how many times, and with what arguments (e.g., verifying `requirePassword` was called or `delete` was never called).

Q5: Why did you use `ReflectionTestUtils` in your unit tests?

A: Our entities have private `id` fields managed by JPA with no public setters. To test methods that expect an existing ID without running Hibernate, we use `ReflectionTestUtils.setField(entity, "id", 10L)`.

Q6: What is Spring `MockMvc`?

A: It is Spring's web testing framework. It simulates HTTP requests and feeds them directly to the `DispatcherServlet` in RAM, testing controllers, validation, and security without opening a real network socket.

Q7: Why annotate integration tests with `@Transactional`?

A: In test classes, `@Transactional` automatically rolls back the database transaction when each test finishes. This ensures tests never pollute the database or affect subsequent tests.

Q8: How do your tests run without MySQL installed?

A: Our test profile in `src/test/resources/application.yml` uses the in-memory H2 database configured in MySQL dialect mode: `jdbc:h2:mem:testdb;MODE=MySQL`.

Q9: How do your integration tests simulate logged-in users?

A: We use Spring Security Test's `httpBasic("admin", "admin123")` post-processor, which adds HTTP Basic authentication credentials to the simulated request.

Q10: What is the purpose of testing unauthenticated access?

A: It verifies that our Spring Security filter chain is properly configured and that endpoints are not accidentally left publicly accessible without credentials (returning `401 Unauthorized`).

Q11: What is Continuous Integration (CI)?

A: CI is the practice of automatically integrating and validating code changes into a central repository. Every commit triggers an automated build and test runner to detect defects immediately.

Q12: Where is the GitHub Actions workflow configured?

A: Inside the repository at `.github/workflows/ci.yml` using YAML syntax.

Q13: What triggers the GitHub Actions workflow?

A: The `on: push: branches: [ "main" ]` and `on: pull_request: branches: [ "main" ]` directives trigger the build on any push or pull request to the main branch.

Q14: What is `ubuntu-latest` in the workflow?

A: It specifies the GitHub-hosted runner OS environment—currently Ubuntu 24.04 LTS.

Q15: Why did you upgrade `checkout` and `setup-java` to `v5`?

A: Version 4 targeted Node.js 20, which is deprecated on GitHub Actions runners. Version 5 targets Node.js 24, eliminating all runner deprecation warnings.

Q16: Why use the Maven wrapper (`mvnw`) instead of installed `mvn`?

A: `mvnw` downloads and runs the exact Maven version defined in `.mvn/wrapper/maven-wrapper.properties`, guaranteeing identical build behavior regardless of what the runner or developer has installed.

Q17: What does the `-B` flag in `./mvnw -B test` do?

A: It enables *Batch Mode*, which prevents interactive prompts and suppresses animated terminal progress lines, producing clean and deterministic CI logs.

Q18: What does `cache: maven` do in GitHub Actions?

A: It caches the `~/.m2/repository` directory between builds. Instead of downloading all Spring Boot JARs on every commit, Maven reuses the cached dependencies, saving 30–40 seconds per build.

Q19: How do you verify that passwords are never returned in API responses?

A: In `TeacherIntegrationTest`, we assert `.andExpect(jsonPath("$.password").doesNotExist())`. Additionally, our response DTO records simply omit the password field.

Q20: What happens if a developer pushes code that breaks a test?

A: Maven exits with code 1, the GitHub Actions job fails with a red cross, GitHub blocks merging the pull request, and the developer receives an immediate failure notification.

10. 1-Minute Lab Presentation Pitch

• Recommended Script to Present to Your Teacher

"Sir, for this lab I implemented an enterprise-grade automated testing and CI/CD strategy following the Test Pyramid:

***1. Unit Tests:** I created **6 isolated unit tests** in `AdminServiceTest` and `StudentServiceTest` using JUnit 5 and Mockito. These test our business rules—such as preventing deletion of the last admin account and rejecting duplicate student IDs—running in just **0.22 seconds** without booting Spring.*

***2. Integration Tests:** I created **3 full-slice integration tests** in `TeacherIntegrationTest` using Spring `MockMvc` and an in-memory H2 database. These test the complete HTTP CRUD lifecycle, confirm password hashes are never leaked, and verify 401 Unauthorized and 409 Conflict handling across our Spring Security filters.*

***3. CI Pipeline:** I configured a **GitHub Actions CI workflow** targeting JDK 21 on Ubuntu runners with dependency caching. The workflow triggers on every push and pull request, executes `./mvnw -B test`, and verifies all **25 tests pass green in under 30 seconds** with zero warnings."*

"""